

Section 1 / 14

[Go to Questions](#) ↓

Introduction to Windows

As a penetration tester, it is important to have knowledge of a wide variety of technologies. A thorough understanding of Windows and Linux operating systems is beneficial in a wide range of assessment types. The majority of systems that we encounter during assessments, whether on-premise or in the cloud, will be based on these two operating systems. It is important to understand how to attack and defend these operating systems and how they can each be used as a platform to perform further penetration testing activities.

The Windows Operating System

Microsoft first introduced the Windows operating system on November 20, 1985. The first version of Windows was a graphical operating system shell for MS-DOS. Later versions of Windows Desktop introduced the Windows File Manager, Program Manager, and Print Manager programs.

Windows 95 was the first full integration of Windows and DOS and offered built-in Internet support for the first time. This version also debuted the Internet Explorer web browser. Since the initial version, there have been over a dozen versions of Windows released, such as Windows XP, Vista, and 8, up to the current version: Windows 11. Over time, Microsoft has offered various editions of

each Windows Desktop release catering to everyone from casual consumers to enterprise customers.

Windows Server was first released in 1993 with the release of Windows NT 3.1 Advanced Server. Windows NT saw several updates over the years, adding in technologies such as Internet Information Services (IIS), various networking protocols, Administrative Wizards to facilitate admin tasks, and more. With the release of Windows 2000, Microsoft debuted Active Directory, originally intended to help sysadmins set up file sharing, data encryption, VPNs, etc. Windows Server 2000 also included the Microsoft Management Console (MMC) and supported dynamic disk volumes.

Windows Server 2003 came next with server roles, a built-in firewall, the Volume Shadow Copy Service, and more. Windows Server 2008 included failover clustering, Hyper-V virtualization software, Server Core, Event Viewer, and major enhancements to Active Directory. Over the years, Microsoft released further Server versions, including Server 2012, Server 2016, and most recently, Server 2019. This latest version added support for Kubernetes, Linux containers, and more advanced security features.

As new versions of Windows are introduced, older versions are deprecated and no longer receive Microsoft updates (unless a long-term support contract is purchased in some cases). Windows Server 2008 and 2012 reached end of life for security updates on January 14, 2020. Currently, only Server 2012 R2 and later are in support. However, Microsoft has released out-of-band patches for earlier versions of Windows in the past few years due to the discovery of the critical SMBv1 vulnerability (EternalBlue).

Many versions of Windows are now deemed "legacy" and are no longer supported. Organizations often find themselves running various older operating systems to support critical applications or

due to operational or budgetary concerns. An assessor needs to understand the differences between versions and the various misconfigurations and vulnerabilities inherent to each.

Windows Versions

The following is a list of the major Windows operating systems and associated version numbers:

Operating System Names	Version Number
Windows NT 4	4.0
Windows 2000	5.0
Windows XP	5.1
Windows Server 2003, 2003 R2	5.2
Windows Vista, Server 2008	6.0
Windows 7, Server 2008 R2	6.1
Windows 8, Server 2012	6.2
Windows 8.1, Server 2012 R2	6.3
Windows 10, Server 2016, Server 2019	10.0

We can use the `Get-WmiObject` cmdlet to find information about the operating system. This cmdlet can be used to get instances of WMI classes or information about available WMI classes. There are a variety of ways to find the version and build number of our system. We can easily obtain this information using the `win32_OperatingSystem` class, which shows that we are on a Windows 10 host, build number 19041.

```
powershell-session
PS C:\htb> Get-WmiObject -Class win32_OperatingSystem | select Version,BuildNumber

Version      BuildNumber
-----      -
10.0.19041 19041
```

Some other useful classes that can be used with `Get-WmiObject` are `Win32_Process` to get a process listing, `Win32_Service` to get a listing of services, and `Win32_Bios` to get **Basic Input/Output System** (`BIOS`) information. The BIOS is firmware installed on a computer's motherboard that controls the computer's essential functions, such as power management, input/output interfaces, and system configuration. We can use the `ComputerName` parameter to get information about remote computers. `Get-WmiObject` can be used to start and stop services on local and remote computers, and more. Further information about the cmdlet can be found [here](#) and [here](#).

Accessing Windows

Local Access Concepts

If you are reading these words right now, you have local access to a computer of some kind. Be it a smartphone, tablet, laptop, Raspberry Pi, or Desktop. Local access is the most common way to access any computer, including computers running Windows. **Input** is likely happening through a keyboard, trackpad &/or mouse. **Output** is coming from the display screen(s). Organizations with office space where employees work on a day-to-day basis build security policies and security controls around the idea that their employees are working in dedicated workspaces on computers owned by the organization. It is becoming more common to see organizations increasing their support for remote work with their non-technical and technical workforces. This is not a new reality for technical professionals working in IT, Software Development & Infosec. On any given day, a technical professional could be accessing multiple machines locally and remotely. With that, let's discuss the concept of remote access.

Remote Access Concepts

Remote Access is accessing a computer over a network. Local access to a computer is needed before one can access another computer remotely. There are countless methods for remote access. In this module, we will mainly use remote access methods to connect to and interact with Windows operating systems. Advances in Networking & Internet technologies have given birth to entire industries that rely completely on remote access & remote administration of computer systems.

Consider **MSPs** & **MSSPs**, both industries are primarily dependent on managing their client's computer systems remotely. This functionality allows them to centralize management, standardize what technologies are used, automate numerous tasks, enable remote work arrangements and allow for quick response time when issues surface, or potential security threats emerge. Remote access is not just limited to MSPs & MSSPs. Organizations with IT, Software Development &/or Security teams use remote access methods daily to build applications, manage servers and

administer employee workstations. Some of the most common remote access technologies include but aren't limited to:

- Virtual Private Networks (VPN)
- Secure Shell (SSH)
- File Transfer Protocol (FTP)
- Virtual Network Computing (VNC)
- Windows Remote Management (or PowerShell Remoting) (WinRM)
- Remote Desktop Protocol (RDP)

We will focus primarily on using RDP in this module.

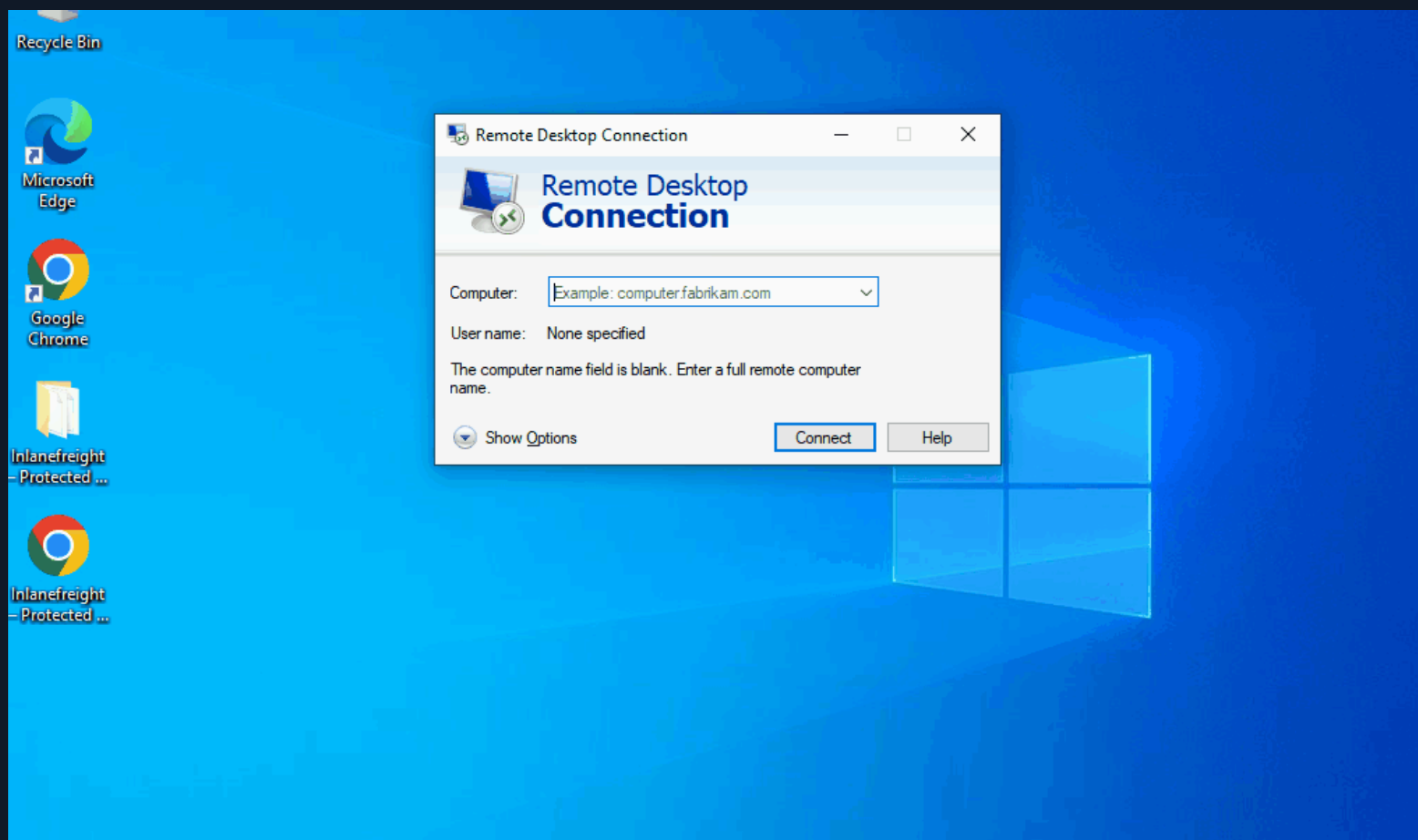
Remote Desktop Protocol (RDP)

RDP uses a client/server architecture where a client-side application is used to specify a computer's target IP address or hostname over a network where RDP access is enabled. The target computer where RDP remote access is enabled is considered the server. It is important to note that RDP listens by default on logical port **3389**. Keep in mind that an IP address is used as a logical identifier for a computer on a network, and a logical port is an identifier assigned to an application. In simpler terms, we could consider a network subnet a street in a town (the corporate network), an IP address in that subnet assigned to a host as a house on that street, and logical ports as windows/doors that can be used to access the house.

Once a request (encapsulated inside a packet) has reached a destination computer via its IP address, the request will be directed to an application hosted on the computer based on the port specified in that request (included as a header inside a packet). IP addressing and protocol encapsulation are covered in greater detail in the module [Introduction to Networking](#). From a

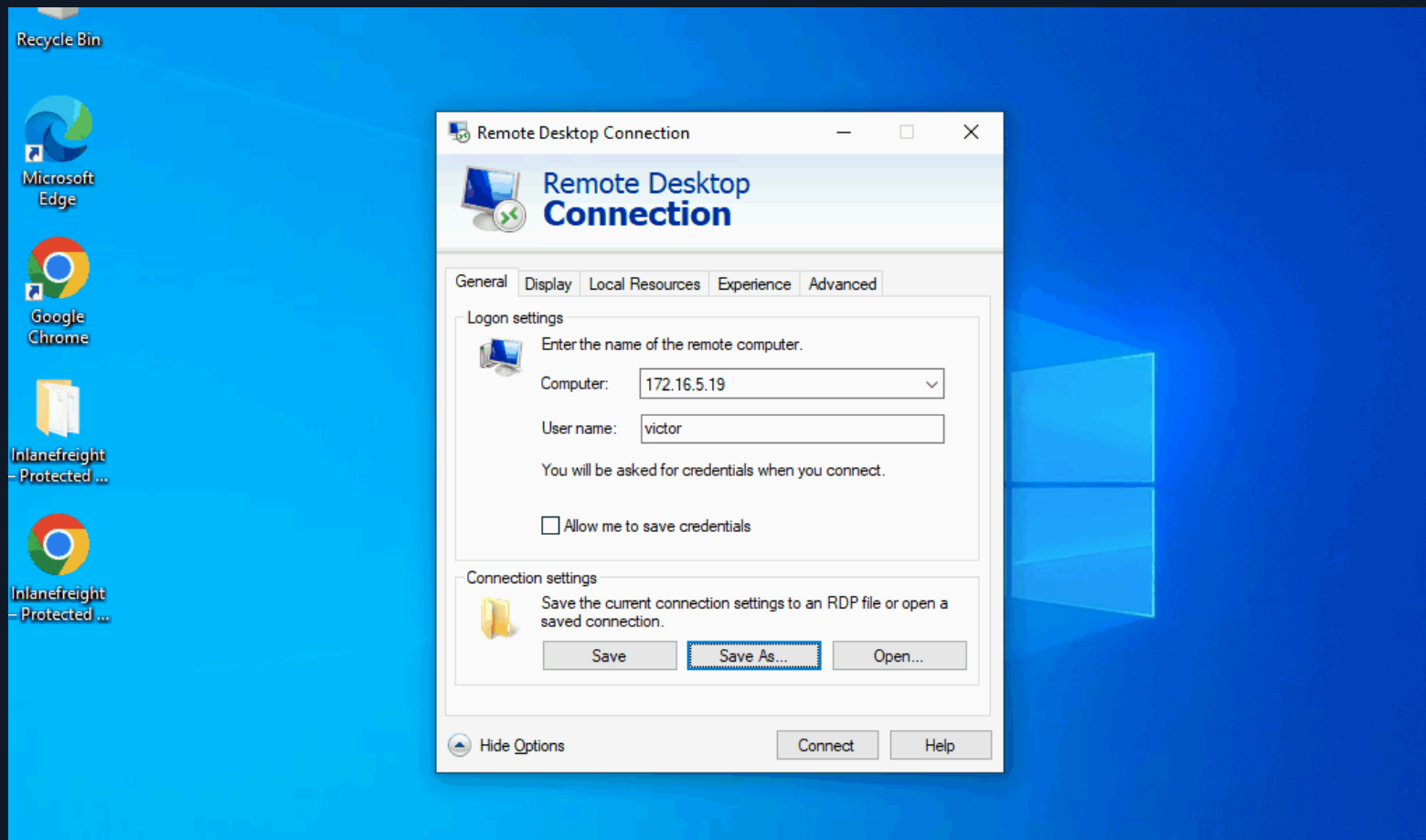
networking perspective, in this module, we only need to understand that every computer has an IP address assigned to communicate over a network, and applications hosted on target computers listen on specific logical ports.

We can use RDP to connect to a Windows target from an attack host running Linux or Windows. If we are connecting to a Windows target from a Windows host, we can use the built-in RDP client application called **Remote Desktop Connection** (**mstsc.exe**). Check out the clip below to see basic usage:



For this to work, remote access must already be **allowed** on the target Windows system. By default, remote access is not allowed on Windows operating systems. The HTB Academy team has configured many of our Windows targets to permit RDP access once connected to the Academy labs via VPN.

Remote Desktop Connection also allows us to save connection profiles. This is a common habit among IT admins because it makes connecting to remote systems more convenient.

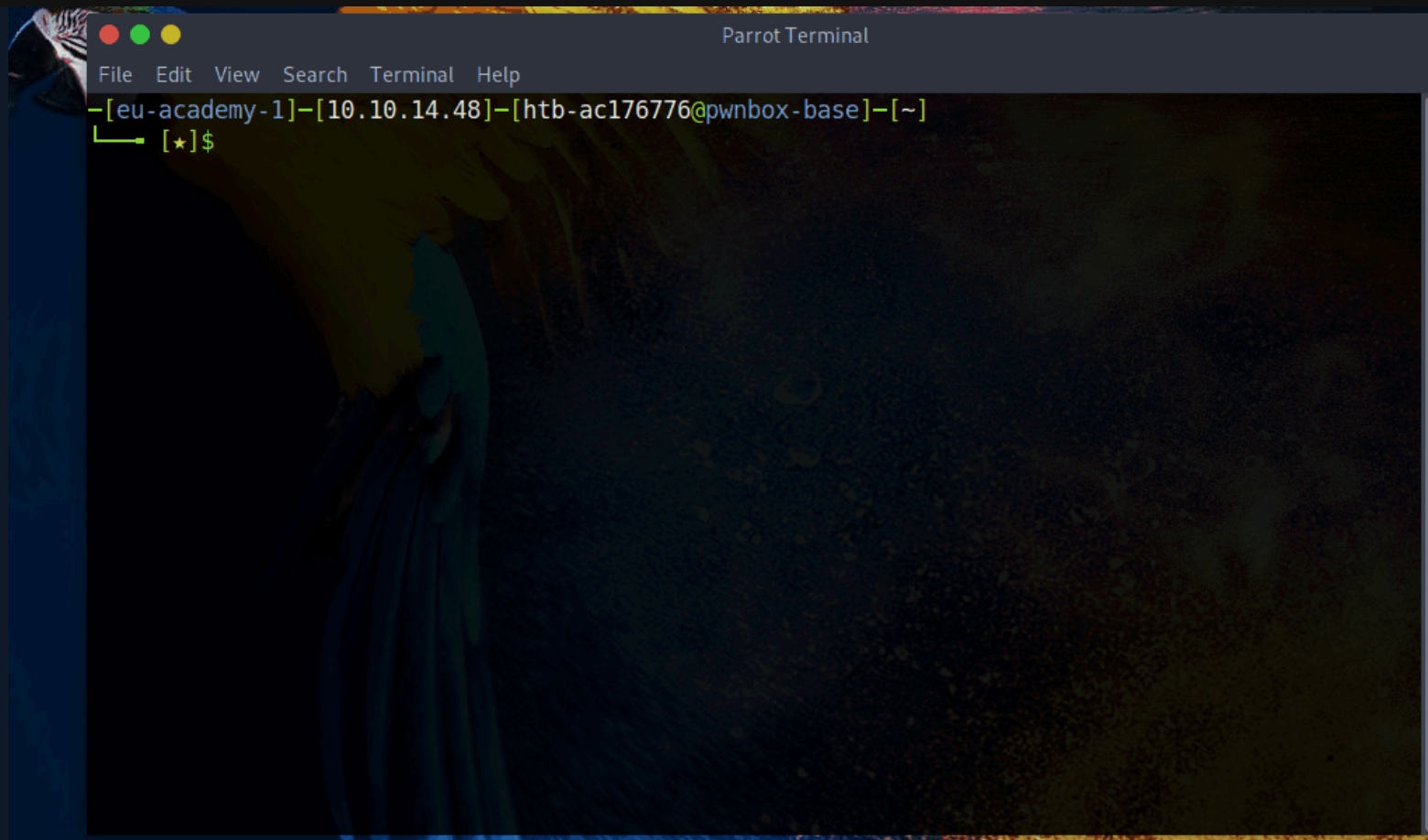


As pentesters, we can benefit from looking for these saved Remote Desktop Files (`.rdp`) while on an engagement.

Many other Remote Desktop client applications exist, some of which are listed in this Microsoft article called [Remote Desktop clients](#). We will not cover every Remote Desktop client application in this module.

Using xfreerdp

From a Linux-based attack host we can use a tool called `xfreerdp` to remotely access Windows targets. You will notice that we use xfreerdp across multiple modules because of its ease of use, feature set, command line utility, and efficiency. Check out the clip below to see basic usage from Pwnbox:



Remember that we can also copy and paste in xfreerdp commands in the command line, so we do not need to enter options manually. There are several options available to us with xfreerdp, such as drive redirection to be able to transfer files to/from the target host, which are worth practicing and we will cover in other modules within HTB Academy.

Other RDP clients exist, such as **Remmina** and **rdesktop**, and we encourage you to experiment with others and see what works best for you. Now that we have covered these concepts let's apply them by spawning the target below and connecting to it using RDP with the credentials provided.

Connecting to the Windows Target

Connect via Remote Desktop (RDP) using the following command:

```
athane@htb[/htb]$ xfreerdp /v:<targetIp> /u:htb-student /p:Password
```

shellsession

Note: It may take 1-2 minutes for your target instance to spawn.

Connect to HTB

Target(s)

Spawn the target system to get IPs and answer questions

🔌 Spawn the target system



Enable step-by-step solutions ⓘ PRO

Question 1

XP +20 ^

What is the Build Number of the target workstation?

RDP to with user "**htb-student**" and password "**Academy_WinFun!**"

Write your answer

Submit

Question 2

XP +20 ^

Which Windows NT version is installed on the workstation? (i.e. Windows X - case sensitive)

Write your answer

Submit



Windows Fundamentals

0%



Mark Complete & Next